Algorithm Design and Analysis

Lecturer: Nguyễn Thị Hồng Minh - Đặng Thị Oanh
Trần Bá Tuấn - Trần Anh Minh

HUS - HKII, 2025-2026

Assignment 4

§ Divide and Conquer §

Section 1: Objectives

- Students need to understand the basic concepts of the divide and conquer technique and the divide and conquer algorithmic scheme.
- Students understand illustrative examples and implement experiments using divide and conquer algorithms, and then provide comments and conclusions.
- Students clearly understand how to solve recurrence relations using the substitution method and the master theorem to determine algorithm complexity.
- Students explore several forms of the "simplify and conquer" technique that were presented in the theoretical lecture.

Section 2: Practice

(1) Quy cách nộp bài

- Nộp bài bằng file văn bản hoặc ảnh chụp scan thành PDF nếu bài làm viết tay ra giấy (khuyến khích sinh viên thực hiện trình bày bài làm bằng Latex).
- Chương trình viết bằng 1 trong 3 ngôn ngữ Java, C/C++, Python. Mã nguồn chương trình, file dữ liệu (nếu có) được đặt trong cùng thư mục, kèm theo hướng dẫn cách thực hiện chương trình nếu cần.
- Tất cả các file liên quan tới bài tập để trong thư mục tên Hw4_MaSinhVien_Hovaten, thư mục được nén thành file.zip cùng tên thư mục.
- Khi hết hạn nộp bài, các bài làm sẽ công bố để thực hiện chấm chéo bài tập.
- Sinh viên không nộp bài sẽ nhận điểm 0 bài tập tuần.
- Sinh viên CÓ GIAN LẬN trong nộp bài tập sẽ bị ĐÌNH CHỈ môn học (điểm 0 cho tất cả các điểm thành phần).

(2) Exercises

- Trình bày sơ lược về chia để trị, lược đồ giải thuật chia để trị cho sinh viên.
- **(Giới thiệu) Master Theorem for Divide and Conquer Recurrences**
  Công thức có dạng:

$$T(n) = aT(\frac{n}{b}) + \Theta(n^k log^p n)$$

  với $a \geq 1$, $b > 1$, $k \geq 0$ và số thực p.

  1. Nếu $a > b^k$ thì $T(n) = \Theta(n^{\log_b a})$
  2. Nếu $a = b^k$ thì
     a) Nếu $p > -1$ thì $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
     b) Nếu $p = -1$ thì $T(n) = \Theta(n^{\log_b a} \log \log n)$
     c) Nếu $p < -1$ thì $T(n) = \Theta(n^{\log_b a})$
  3. Nếu $a < b^k$ thì
     a) Nếu $p \geq 0$ thì $T(n) = \Theta(n^k \log^p n)$
     b) Nếu $p < 0$ thì $T(n) = \Theta(n^k)$

Example 1. *Let us consider an algorithm A which solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time. What is the complexity of this algorithm?*

Example 2. *Similar to Problem-1, an algorithm B solves problems of size n by recursively solving two subproblems of size n - 1 and then combining the solutions in constant time. What is the complexity of this algorithm?*

Example 3. *Againt similar to Problem-1, another algorithm C solves problems of size n by dividing them into nine subproblems of size* $\frac{n}{3}$*, recursively solving each subproblem and then combining the solution in* $O(n^2)$ *time. What is the complexity of this algorithm?*

Example 4. *Write a recurrence and solve it*

```
public void fuction (int n){
    if (n > 1){
        System.out.println("*");
        function(n/2);
        function(n/2);
    }
}
```

Example 5. ***Familiarize yourself with the formula introduced above.***

a) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$

b) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

c) $T(n) = 16T\left(\frac{n}{4}\right) + n$

d) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{logn}$

e) $T(n) = \sqrt{2}T\left(\frac{n}{2}\right) + \log n$

f) $T(n) = 3T\left(\frac{n}{3}\right) + \sqrt{n}$

Exercise 1. *Xác định độ phức tạp của công thức sau:*

a) $T(n) = 2T(\sqrt{n}) + \log n$.

b) $T(n) = T(\sqrt{n}) + 1$. *Gợi ý: Áp dụng ý a.*

Exercise 2. ***Write programs that apply the divide and conquer technique to solve the following problems:***

1. *Given an integer array, find the elements with the maximum and minimum values.*
2. *Given two square matrices A and B of order n, where n is a power of 2. Compute the product of the two matrices using the standard divide and conquer method and the Strassen algorithm.*
3. *Use the QuickSort algorithm to sort the values in an array of numbers in a specified order (either ascending or descending).*
4. *Use the MergeSort algorithm to sort the values in an array of numbers in a specified order (either ascending or descending).*
5. *Check for Majority Element in a sorted array.*
6. *Median of two sorted arrays of different sizes.*

Provide an evaluation (it is encouraged to use charts/graphs) of how the program's execution time grows with respect to the input data size.

Exercise 3. ***Problem formulation, algorithm design, analysis, and implementation***

Formulate at least two problems, analyze them, design the corresponding algorithms, analyze the algorithms, and implement programs to illustrate the divide and conquer technique or another related paradigm (decrease and conquer or transform and conquer).

Some suggested problems:

- Closest Pair of Points problem (Anany's book, Sec. 5.5)
- Convex Hull problem (Anany's book, Sec. 5.5)
- Fake Coin problem (Anany's book, Sec. 4.4)
- Gaussian Elimination: solving systems of equations, computing determinants, and finding matrix inverses (Anany's book, Sec. 6.2)

Exercise 4. ***(Stock Pricing Problem)*** *Consider the stock price of CareerMonk.com in n consecutive days. That means the input consists of an array with stock prices of the company. We know that the stock price will not be the same on all the days. In the input stock prices there may be dates where the stock is high when we can sell the current holdings, and there may be days when we can buy the stock. Now our problem is to find the day on which we can buy the stock and the day on which we can sell the stock so that we can make maximum profit.*

Exercise 5. *We are testing "unbreakable" laptops and our goal is to find out how unbreakable they really are. In particular, we work in an n-story building and want to find out the lowest floor from which we can drop the laptop without breaking it (call this "the ceiling"). Suppose we are given two laptops and want to find the highest ceiling possible. Give an algorithm that minimizes the number of tries we need to make $f(n)$ (hopefully, $f(n)$ is sub-linear, as a linear $f(n)$ yields a trivial solution).*

Exercise 6. *Given a square floor of size $2^n \times 2^n$. One cell is reserved for drainage. Find a way to tile the floor with L-shaped tiles so that the entire floor is covered, except for the square cell used for drainage.*